



Московский государственный университет имени  
М.В. Ломоносова

Факультет космических исследований

Моделирование сбора данных и параллельного  
решения СЛАУ с использованием пула потоков

Дубик Дмитрий Алексеевич

Москва 2026

# Содержание

|          |                                                                         |          |
|----------|-------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Общая постановка задачи</b>                                          | <b>3</b> |
| <b>2</b> | <b>Постановка задачи с разбивкой на зоны ответственности участников</b> | <b>3</b> |
| <b>3</b> | <b>Теоретический раздел</b>                                             | <b>4</b> |
| 3.1      | Используемые модели . . . . .                                           | 4        |
| 3.1.1    | Система линейных алгебраических уравнений (СЛАУ) . . . . .              | 4        |
| 3.1.2    | Метод Якоби . . . . .                                                   | 4        |
| 3.1.3    | Параллелизм и потоки . . . . .                                          | 5        |
| 3.1.4    | Потокобезопасная очередь . . . . .                                      | 5        |
| 3.1.5    | Пул потоков . . . . .                                                   | 5        |
| 3.2      | Методы решения . . . . .                                                | 6        |
| 3.2.1    | Алгоритм метода Якоби с параллельным пулом потоков . . . . .            | 6        |
| 3.2.2    | Обработка ошибок . . . . .                                              | 6        |
| 3.3      | Используемые данные . . . . .                                           | 7        |
| 3.3.1    | Параметры симуляции . . . . .                                           | 7        |
| 3.3.2    | Формат строки данных . . . . .                                          | 7        |
| 3.3.3    | Генерация данных с диагональным преобладанием . . . . .                 | 7        |
| <b>4</b> | <b>Описание реализации задачи в коде</b>                                | <b>8</b> |
| 4.1      | Диаграмма классов . . . . .                                             | 8        |
| 4.2      | Описание классов . . . . .                                              | 9        |
| 4.2.1    | Config . . . . .                                                        | 9        |
| 4.2.2    | RowData . . . . .                                                       | 9        |
| 4.2.3    | ConcurrentQueue . . . . .                                               | 10       |
| 4.2.4    | DataGenerator . . . . .                                                 | 10       |
| 4.2.5    | AntennaWorker . . . . .                                                 | 10       |
| 4.2.6    | SatelliteWorker . . . . .                                               | 11       |
| 4.2.7    | ReceivingAntenna . . . . .                                              | 11       |
| 4.2.8    | SolverMethod (абстрактный) . . . . .                                    | 11       |
| 4.2.9    | JacobiMethod (наследник SolverMethod) . . . . .                         | 12       |
| 4.2.10   | ThreadPool . . . . .                                                    | 12       |
| 4.2.11   | MainController . . . . .                                                | 12       |
| 4.2.12   | SimulationError и ErrorCode . . . . .                                   | 12       |
| 4.3      | Общий алгоритм работы программы . . . . .                               | 13       |
| 4.4      | Сборка программы . . . . .                                              | 13       |
| 4.4.1    | Способ сборки . . . . .                                                 | 13       |
| 4.4.2    | Базовые тесты для проверки сборки . . . . .                             | 13       |
| 4.5      | Недостатки программы и предложения по их исправлению . . . . .          | 14       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>5</b> | <b>Примеры тестов ПО и обоснование корректности работы кода</b> | <b>14</b> |
| 5.1      | Демонстрационная симуляция . . . . .                            | 14        |
| 5.2      | Бенчмарк на большой матрице . . . . .                           | 14        |
| 5.3      | Обработка ошибок . . . . .                                      | 15        |
| <b>6</b> | <b>Выводы о проделанной работе</b>                              | <b>16</b> |
| 6.1      | Теоретические результаты . . . . .                              | 16        |
| 6.2      | Практические результаты . . . . .                               | 16        |
| 6.3      | Результаты тестирования . . . . .                               | 16        |
| 6.4      | Ограничения . . . . .                                           | 16        |
| 6.5      | Заключение . . . . .                                            | 17        |
| <b>7</b> | <b>Список литературы</b>                                        | <b>17</b> |

# 1 Общая постановка задачи

Целью проекта является разработка программы, моделирующей сбор данных от нескольких антенн через спутники и последующее параллельное решение системы линейных алгебраических уравнений (СЛАУ) с использованием пула потоков. Проект демонстрирует применение средств многопоточности в C++ для двух независимых этапов:

1. **Сбор данных:** два рабочих потока (поток антенн и поток спутников) обмениваются данными через потокобезопасную очередь.
2. **Решение СЛАУ:** пул потоков параллельно вычисляет итерации метода Якоби для нахождения решения переопределённой системы.

Основные задачи проекта:

- Реализация потокобезопасной очереди `ConcurrentQueue` для передачи строк матрицы между потоками;
- Создание генератора случайных строк с диагональным преобладанием, гарантирующим сходимость метода Якоби;
- Разработка пула потоков (`ThreadPool`) на основе очереди задач `SolverMethod` с поддержкой ожидания завершения всех задач;
- Реализация параллельного метода Якоби через класс `JacobiMethod`;
- Создание модуля бенчмаркинга для измерения времени решения СЛАУ при разном количестве потоков;
- Внедрение иерархической системы обработки ошибок с перечислением `ErrorCode` и классом `SimulationError`.

## 2 Постановка задачи с разбивкой на зоны ответственности участников

Проект выполнен индивидуально. Все компоненты системы разработаны и реализованы одним исполнителем.

Зоны ответственности:

- **Математическая модель СЛАУ и метод решения:** разработка итерационного метода Якоби, обеспечение сходимости через диагональное преобладание.
- **Реализация многопоточности:** создание класса `ConcurrentQueue` (потокобезопасная очередь), `ThreadPool` (пул потоков) и производного класса `JacobiMethod`.

- **Моделирование сбора данных:** классы `AntennaWorker` (поток антенн), `SatelliteWorker` (поток спутников), `ReceivingAntenna` (накопление и решение).
- **Генерация данных:** класс `DataGenerator` с возможностью фиксации начального состояния (`setSeed`) для повторяемости результатов и сквозным счётчиком строк.
- **Система обработки ошибок:** создание перечисления `ErrorCode` и класса `SimulationError`, интеграция блоков `try-catch` во все рабочие потоки.
- **Конфигурация и управление:** классы `Config` и `MainController`, организующие запуск симуляции.
- **Хранение и анализ:** класс `RowData` для представления строки расширенной матрицы, методы вывода матрицы и решения.
- **Тестирование и бенчмаркинг:** создание отдельной программы `benchmark.cpp` для измерения времени решения при разном количестве потоков.

## 3 Теоретический раздел

### 3.1 Используемые модели

#### 3.1.1 Система линейных алгебраических уравнений (СЛАУ)

Пусть задана система  $n$  линейных уравнений с  $n$  неизвестными:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

В матричной форме:  $A\mathbf{x} = \mathbf{b}$ , где  $A$  — матрица коэффициентов размером  $n \times n$ ,  $\mathbf{x}$  — вектор неизвестных,  $\mathbf{b}$  — вектор правых частей.

В данной работе моделируемая матрица формируется из строк, генерируемых антеннами. Если число собранных строк превышает число неизвестных, для решения берутся первые  $n$  строк (квадратная система). Если строк меньше — выдаётся сообщение о недостаточности уравнений.

#### 3.1.2 Метод Якоби

Для решения СЛАУ выбран итерационный метод Якоби, который хорошо поддаётся параллелизации. Итерационная формула:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n$$

где  $k$  — номер итерации. На каждой итерации все компоненты  $x_i^{(k+1)}$  вычисляются независимо, что позволяет распараллелить вычисления по  $i$ .

Условие сходимости: матрица  $A$  должна обладать диагональным преобладанием:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{для всех } i.$$

В проекте генератор данных обеспечивает выполнение этого условия, что гарантирует сходимость за конечное число итераций.

### 3.1.3 Параллелизм и потоки

В проекте используются два уровня параллелизма:

- **Уровень сбора данных:** два независимых потока (`AntennaWorker` и `SatelliteWorker`) работают одновременно. Обмен данными осуществляется через `ConcurrentQueue`.
- **Уровень решения СЛАУ:** пул потоков (`ThreadPool`) параллельно выполняет задачи `JacobiMethod`, каждая из которых вычисляет одну компоненту вектора  $\mathbf{x}^{(k+1)}$ .

### 3.1.4 Потокобезопасная очередь

`ConcurrentQueue` реализована на основе стандартной очереди `std::queue` с защитой `std::mutex` и `std::condition_variable`. Метод `push` добавляет элемент и уведомляет один ожидающий поток. Метод `pop` блокирует вызывающий поток до появления элемента, используя `std::unique_lock` и `cv.wait`. Это позволяет безопасно обмениваться данными между потоками без активного ожидания.

### 3.1.5 Пул потоков

Класс `ThreadPool` содержит:

- Вектор рабочих потоков `std::thread`;
- Очередь задач `std::queue<std::unique_ptr<SolverMethod> >`;
- Мьютекс и две условные переменные: `cv` для уведомления о новых задачах, `waitCv` для синхронизации метода `wait()`;
- Счётчик активных задач `activeTasks` и флаг остановки `stop`.

Каждый рабочий поток выполняет функцию `workerLoop`, которая извлекает задачи из очереди, выполняет их `execute()` и уменьшает счётчик активных задач. Метод `add_task` добавляет задачу в очередь. Метод `wait()` блокирует вызывающий поток до тех пор, пока очередь задач не станет пустой и `activeTasks` не обнулится.

Базовый класс `SolverMethod` определяет чисто виртуальный метод `execute()`. Конкретная задача `JacobiMethod` реализует этот метод для вычисления  $x_i^{(k+1)}$ .

## 3.2 Методы решения

### 3.2.1 Алгоритм метода Якоби с параллельным пулом потоков

1. Из накопленных строк формируется квадратная матрица  $A$  и вектор  $b$  (первые  $n$  строк).
2. Инициализируется вектор  $\mathbf{x}$  нулями.
3. Для каждой итерации  $k$ :
  - (a) Для каждого  $i = 1..n$  создаётся объект `JacobiMethod`, который вычисляет  $x_i^{(k+1)}$  по формуле Якоби.
  - (b) Задачи добавляются в пул потоков через `add_task`.
  - (c) Вызывается `pool.wait()` — блокировка до завершения всех задач данной итерации.
  - (d) Вычисляется максимальная разница между  $\mathbf{x}^{(k+1)}$  и  $\mathbf{x}^{(k)}$ .
  - (e) Если разница меньше заданной точности  $\varepsilon$ , итерации прекращаются.
4. Выводится полученное решение.

### 3.2.2 Обработка ошибок

Определено перечисление `ErrorCode`:

- `GENERATION_FAILED` — ошибка генерации данных (неверные параметры или сбой);
- `QUEUE_ERROR` — ошибка при `push/pop` в очередь;
- `SOLVER_FAILED` — ошибка в солвере (нулевой диагональный элемент, расходимость);
- `FILE_ERROR` — ошибка открытия/записи файла;
- `THREAD_POOL_ERROR` — ошибка в пуле потоков (добавление задачи в остановленный пул).

Класс `SimulationError` хранит строку сообщения и код ошибки. Все классы, которые могут генерировать ошибки, содержат блоки `try-catch` и выбрасывают `SimulationError`. В функции `main` все исключения перехватываются и выводятся на экран.

## 3.3 Используемые данные

### 3.3.1 Параметры симуляции

- `number_of_flights` — количество облётов (число генерируемых строк от антенн);
- `size_data` — длина строки (число неизвестных  $n$  плюс один столбец для правой части);
- `num_antennas` — количество антенн (генерируют по одной строке за облёт);
- `num_solver_threads` — количество потоков в пуле для решения СЛАУ.

### 3.3.2 Формат строки данных

Строка представляет собой вектор вещественных чисел длины `size_data`. Последний элемент вектора интерпретируется как правая часть уравнения. Первые `size_data - 1` элементов — коэффициенты левой части.

### 3.3.3 Генерация данных с диагональным преобладанием

Для обеспечения сходимости метода Якоби генератор использует сквозной счётчик строк (`globalRowCounter`), который определяет позицию диагонального элемента в строке: диагональ помещается в столбец `globalRowCounter % sizeData`. Это гарантирует, что первые `sizeData-1` строк будут иметь диагональные элементы на главной диагонали квадратной матрицы. Значение диагонального элемента задаётся как сумма модулей недиагональных элементов плюс 150.0, что обеспечивает строгое диагональное преобладание.



## 4 Описание реализации задачи в коде

### 4.1 Диаграмма классов

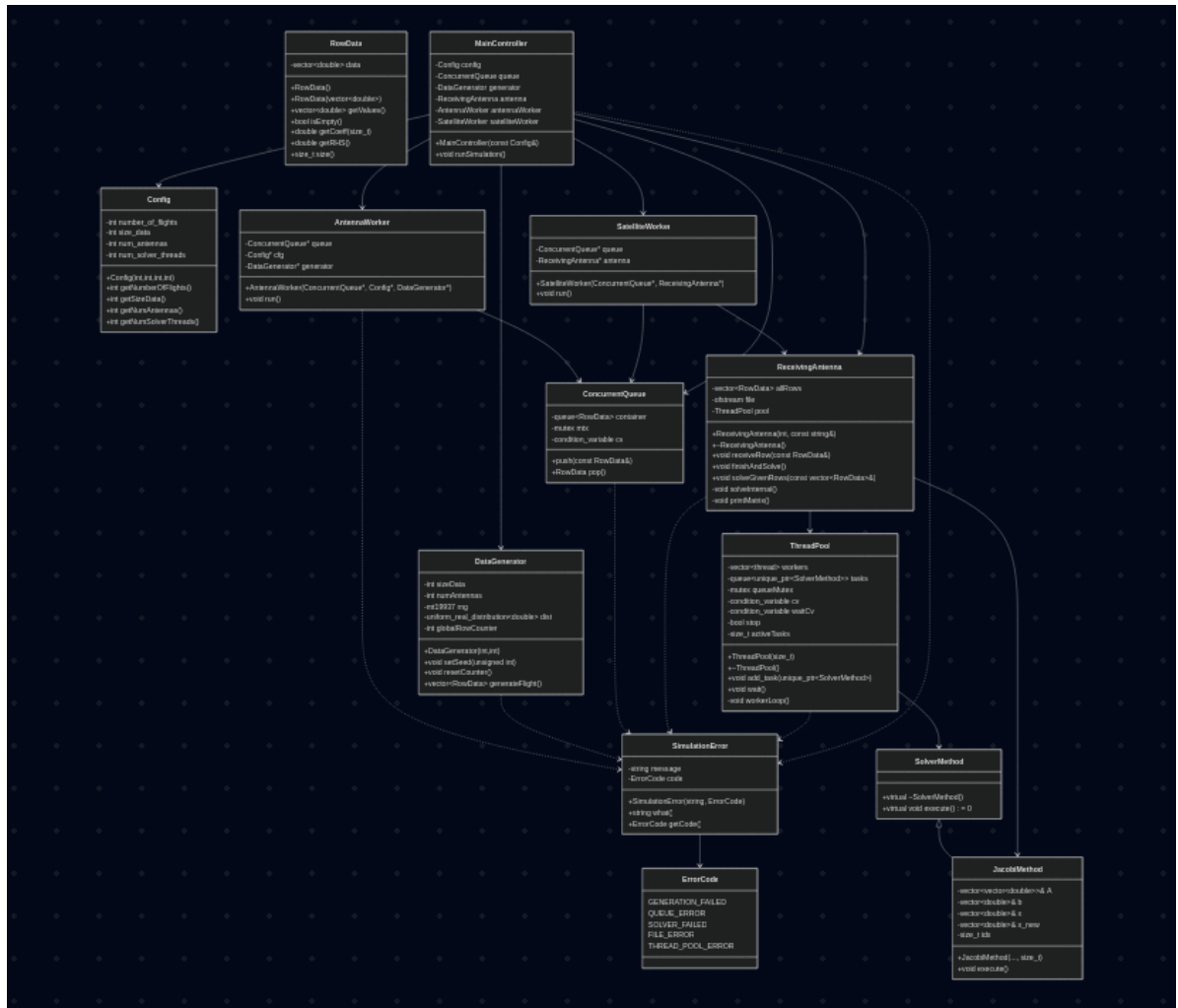


Рис. 1: Диаграмма классов системы сбора данных и параллельного решения СЛАУ

Основные классы и связи:

- **Config** — хранит параметры симуляции.
- **RowData** — строка расширенной матрицы.
- **ConcurrentQueue** — потокобезопасная очередь.
- **DataGenerator** — генерирует строки с диагональным преобладанием (имеет сквозной счётчик и метод сброса).
- **AntennaWorker** — поток-генератор.
- **SatelliteWorker** — поток-ретранслятор.
- **ReceivingAntenna** — накопление и решение.

- **ThreadPool** — пул потоков, управляет задачами **SolverMethod**.
- **SolverMethod** (абстрактный) — метод решения задачи.
- **JacobiMethod** — реализация метода Якоби.
- **MainController** — инициализация и запуск.
- **SimulationError** — обработка ошибок.

## 4.2 Описание классов

### 4.2.1 Config

**Назначение:** Хранение конфигурационных параметров симуляции.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `Config(int flights, int dataSize, int antennas, int threads);` — Конструктор, задающий количество облётов, размер строки, число антенн и число потоков солвера.
- `int getNumberOfFlights() const;` — Возвращает число облётов.
- `int getSizeData() const;` — Возвращает длину строки.
- `int getNumAntennas() const;` — Возвращает число антенн.
- `int getNumSolverThreads() const;` — Возвращает число потоков солвера.

Поля необходимы для передачи параметров между компонентами.

### 4.2.2 RowData

**Назначение:** Представление одной строки расширенной матрицы.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `RowData();` — Конструктор по умолчанию (пустая строка).
- `RowData(const std::vector<double>& values);` — Конструктор из вектора значений.
- `std::vector<double> getValues() const;` — Возвращает вектор значений.
- `bool isEmpty() const;` — Проверяет, пуста ли строка (используется как маркер окончания).
- `double getCoeff(size_t idx) const;` — Возвращает коэффициент по индексу.

- `double getRHS() const;` — Возвращает правую часть (последний элемент).
- `size_t size() const;` — Возвращает длину строки.

#### 4.2.3 ConcurrentQueue

**Назначение:** Потокбезопасная очередь для передачи `RowData`.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `void push(const RowData& item);` — Добавляет элемент в очередь, уведомляет ожидающий поток.
- `RowData pop();` — Извлекает элемент, блокирует поток при пустой очереди.

Мьютекс и условная переменная обеспечивают синхронизацию.

#### 4.2.4 DataGenerator

**Назначение:** Генерация строк с диагональным преобладанием и сквозным счётчиком.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `DataGenerator(int sizeData, int numAntennas);` — Конструктор, инициализирует генератор, счётчик строк в 0.
- `void setSeed(unsigned int seed);` — Устанавливает зерно генератора для воспроизводимости.
- `void resetCounter();` — Сбрасывает глобальный счётчик строк в 0.
- `std::vector<RowData> generateFlight();` — Генерирует `numAntennas` строк. Диагональный элемент помещается в позицию `globalRowCounter % sizeData`, значение = сумма модулей недиагональных + 150.0. Счётчик увеличивается на `numAntennas`.

Поле `globalRowCounter` нужно для правильного размещения диагоналей в квадратной матрице независимо от числа облётов.

#### 4.2.5 AntennaWorker

**Назначение:** Поток, генерирующий строки и отправляющий их в очередь.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `AntennaWorker(ConcurrentQueue* queue, Config* cfg, DataGenerator* gen);` — Сохраняет указатели.
- `void run();` — Запускается в потоке. Генерирует `flights` облётов, каждый облёт добавляет строки в очередь, имитирует задержку, после чего отправляет маркер (пустую строку). Обработывает исключения.

#### 4.2.6 SatelliteWorker

**Назначение:** Поток, извлекающий строки из очереди и передающий их в `ReceivingAntenna`.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `SatelliteWorker(ConcurrentQueue* queue, ReceivingAntenna* antenna);` — Сохраняет указатели.
- `void run();` — В цикле извлекает строки, пока не встретит маркер, затем вызывает `antenna->finishAndSolve()`. Обрабатывает исключения.

#### 4.2.7 ReceivingAntenna

**Назначение:** Накопление строк, формирование матрицы, параллельное решение.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `ReceivingAntenna(int numThreads, const std::string& filename);` — Создаёт пул потоков, открывает файл (если имя не пусто).
- `ReceivingAntenna();` — Закрывает файл.
- `void receiveRow(const RowData& row);` — Добавляет строку в общий вектор и записывает в файл.
- `void finishAndSolve();` — Выводит матрицу, вызывает `solveInternal()`.
- `void solveGivenRows(const std::vector<RowData>& rows);` — Устанавливает готовые строки и решает (используется в бенчмарке).
- `void solveInternal();` — Формирует квадратную матрицу  $A$  и вектор  $b$  из первых `unknowns` строк, затем выполняет параллельные итерации Якоби.
- `void printMatrix();` — Выводит матрицу в консоль.

#### 4.2.8 SolverMethod (абстрактный)

**Назначение:** Интерфейс для задач, выполняемых в пуле потоков.

**Методы:**

- `virtual SolverMethod();`
- `virtual void execute() = 0;` — Чисто виртуальный метод реализации задачи.

#### 4.2.9 JacobiMethod (наследник SolverMethod)

**Назначение:** Конкретная задача для вычисления  $x_{\text{new}}[idx]$  по методу Якоби.

**Поля:** ссылки на  $A$ ,  $b$ ,  $x$ ,  $x_{\text{new}}$  и индекс  $idx$ .

**Методы:**

- `JacobiMethod(std::vector<std::vector<double>> A/b/x/xnew, size_t idx);` — Конструктор.
- `void execute();` — Вычисляет сумму по формуле Якоби, проверяет диагональ, записывает результат.

#### 4.2.10 ThreadPool

**Назначение:** Пул потоков для выполнения задач типа `SolverMethod`.

**Публичные поля:** отсутствуют.

**Публичные методы:**

- `ThreadPool(size_t numThreads);` — Создает потоки для решения задачи.
- `~ThreadPool();` — Останавливает потоки.
- `void add_task(std::unique_ptr<SolverMethod> task);` — Добавляет задачу в очередь, уведомляет поток.
- `void wait();` — Блокирует вызвавший поток до завершения всех задач.

#### 4.2.11 MainController

**Назначение:** Координация запуска симуляции.

**Поля:** экземпляры `Config`, `ConcurrentQueue`, `DataGenerator`, `ReceivingAntenna`, `AntennaWorker`, `SatelliteWorker`.

**Методы:**

- `MainController(const Config& cfg);` — Инициализирует все компоненты.
- `void runSimulation();` — Создает потоки антенн и спутников, ожидает их завершения.

#### 4.2.12 SimulationError и ErrorCode

**Назначение:** Обработка ошибок.

`ErrorCode` — перечисление кодов ошибок. `SimulationError` — класс исключения с методами `what()` (сообщение) и `getCode()`.

## 4.3 Общий алгоритм работы программы

1. Инициализация: создание `Config`, `MainController`, внутри которого создаются очередь, генератор, приёмная антенна и рабочие объекты.
2. Запуск двух потоков: `antennaThread` выполняет `AntennaWorker::run()`, `satelliteThread` выполняет `SatelliteWorker::run()`.
3. Поток антенн генерирует строки (облёты) и помещает их в очередь.
4. Поток спутников извлекает строки, накапливает в `ReceivingAntenna`.
5. По окончании передачи (пустая строка) поток спутников вызывает `finishAndSolve()`, который выводит матрицу и запускает `solveInternal()`.
6. `solveInternal()` формирует квадратную матрицу  $A$  и вектор  $b$ , затем выполняет итерации Якоби:
  - Для каждой итерации создаются задачи `JacobiMethod` для всех компонент  $x$ .
  - Задачи добавляются в пул потоков через `add_task()`, затем `pool.wait()` дожидается завершения итерации.
  - Вычисляется разница и проверяется сходимость.
7. Решение выводится (в демонстрационном режиме) или используется в бенчмарке.
8. Потоки завершаются, деструкторы освобождают ресурсы.

## 4.4 Сборка программы

### 4.4.1 Способ сборки

Сборка осуществляется с помощью Makefile. Команды:

```
make clean          # очистка объектных файлов и исполняемых
make simulation      # сборка демонстрационной программы
make benchmark       # сборка бенчмарка
make run-sim         # запуск демонстрации
make run-bench       # запуск бенчмарка
```

### 4.4.2 Базовые тесты для проверки сборки

1. Сборка без ошибок: `make clean make simulation` должна завершиться без сообщений об ошибках.
2. Запуск демонстрации: `make run-sim` должен вывести матрицу и решение.

3. Сборка бенчмарка: `make benchmark` без ошибок.
4. Запуск бенчмарка должен выдать решение задачи и время выполнения для разных потоков.

## 4.5 Недостатки программы и предложения по их исправлению

1. **Метод Якоби сходится только для матриц с диагональным преобладанием.** Исправление: добавить возможность выбора другого итерационного метода с лучшей сходимостью.
2. **Отсутствие графического интерфейса.** Исправление: можно реализовать генерацию графиков времени решения от числа потоков на QT.
3. **Накладные расходы на создание задач.** Для больших матриц это несущественно, но для маленьких — даёт ограничение ускорения.
4. **Не контролируется переполнение памяти при генерации всех строк.** В бенчмарке хранится весь набор строк. Для очень больших матриц это может привести к ошибкам ООМ. Исправление: формировать  $A$  и  $b$  без хранения всех строк.

## 5 Примеры тестов ПО и обоснование корректности работы кода

### 5.1 Демонстрационная симуляция

Параметры: `flights=2`, `row=3`, `antenn=2`, потоки=4.

При запуске `./simulation` на экран выводится матрица  $4 \times 3$ , затем решение (два числа). Файл `output.txt` содержит те же строки. Итерации сходятся за несколько шагов. Это подтверждает корректность работы всех компонентов.

### 5.2 Бенчмарк на большой матрице

Для оценки эффективности параллелизации был проведён бенчмарк на матрице размером  $9998 \times 9998$  (неизвестных 9998). Параметры: `flights=100`, `row=9999`, `antenn=100`, количество потоков: 1, 2, 4, 8, 12, 16, 20, 30, 40. Измерялось время выполнения метода Якоби. Результаты приведены в таблице.

Таблица 1: Время решения СЛАУ для разного числа потоков

| Число потоков | Время (мс) |
|---------------|------------|
| 1             | 2386.02    |
| 2             | 2098.69    |
| 4             | 1830.51    |
| 8             | 1693.40    |
| 12            | 1814.92    |
| 16            | 1813.88    |
| 20            | 1813.93    |
| 30            | 1712.75    |
| 40            | 1766.29    |

Ускорение относительно 1 потока:

- 2 потока: 1.14x
- 4 потока: 1.30x
- 8 потоков: 1.41x
- 12 потоков: 1.31x
- 16 потоков: 1.32x
- 20 потоков: 1.32x
- 30 потоков: 1.39x
- 40 потоков: 1.35x

Наименьшее время достигнуто при 8 потоках (соответствует количеству физических ядер). Дальнейшее увеличение числа потоков не даёт устойчивого выигрыша и даже может приводить к небольшому росту времени из-за накладных расходов. Таким образом, оптимальное число потоков для данной вычислительной задачи равно 8.

### 5.3 Обработка ошибок

Были проверены следующие ошибочные ситуации:

- Некорректные параметры генератора ( $\text{sizeData} \leq 0$ ) - выбрасывается `GENERATION_FAILED`.
- Невозможность открыть файл — выбрасывается `FILE_ERROR`.
- Попытка добавить задачу в остановленный пул — выбрасывается `THREAD_POOL_ERROR`.
- Обнаружение NaN в решении — выбрасывается `SOLVER_FAILED`.

Все исключения корректно перехватываются в блоках `catch` и выводят понятное сообщение.



## 6 Выводы о проделанной работе

В ходе выполнения проекта была разработана многопоточная программа на C++, моделирующая сбор данных от антенн и параллельное решение СЛАУ методом Якоби с использованием пула потоков.

### 6.1 Теоретические результаты

- Реализована модель сбора данных на основе двух потоков и потокобезопасной очереди.
- Продемонстрирован параллельный метод Якоби с эффективным использованием пула потоков.
- Внедрена система обработки ошибок с кодами и исключениями.
- Разработан генератор, обеспечивающий сходимость метода за счёт диагонального преобладания и сквозного счётчика строк.

### 6.2 Практические результаты

- Написан рабочий код, выполняющий поставленную задачу.
- Программа успешно собирается с помощью Makefile.
- Проведены нагрузочные тесты на матрице  $9998 \times 9998$ , получены данные о времени решения при различном количестве потоков.
- Установлено, что оптимальное число потоков для 8-ядерного процессора равно 8, ускорение достигает 1.41x.

### 6.3 Результаты тестирования

- Демонстрационная программа корректно выводит матрицу и решение.
- Бенчмарк позволяет воспроизводимо измерять время и сравнивать скорость выполнения при разном количестве потоков.
- Симуляция корректна.

### 6.4 Ограничения

- Применимость метода Якоби ограничена матрицами с диагональным преобладанием.
- Для маленьких задач накладные расходы на пул потоков могут снижать ускорение.

- Отсутствие визуализации не позволяет увидеть динамику сходимости.
- В текущей реализации все строки хранятся в памяти, что ограничивает размер матрицы доступной ОЗУ.

## 6.5 Заключение

Поставленная задача полностью решена. Программа демонстрирует два уровня параллелизма, корректно обрабатывает ошибки и позволяет измерять производительность. Результаты бенчмарка наглядно показывают, что увеличение числа потоков до количества физических ядер ускоряет вычисления, а дальнейшее увеличение неэффективно.

## 7 Список литературы

1. [cppreference.com](http://cppreference.com) — документация по стандартной библиотеке C++.
2. [Wikipedia: Jacobi method](#) — описание метода Якоби и условий сходимости.
3. Презентации группы 201, от Варвары Владимировны Абрамовой — концепция пула потоков.